

UNIVERSITÉ CATHOLIQUE DE LOUVAIN

LSINF2275 – DATA MINING AND DECISION MAKING

Project Report

“Markov Decision Processes and Reinforcement Learning”

Academic year 2023-2024

Submission date : 20 May 2024

GROUP : 16

Pietro GAVAZZI (6602 1900)
Francis JACOBS (1299 1800)
Alex WLODAWER (5028 2000)

DATE
INFO2
DATS2M/G

PROFESSOR :
Marco SAERENS

TEACHING ASSISTANTS :
Pierre LELEUX
Nicolas SZELAGOWSKI

1. Part I

1.1. Introduction

In this part, we implemented a code that models the Snakes and Ladders game and that returns a choice of which dice to play on each tile of the layout.

For this part, we will be in the context of a one person game, on a 15 tiles board. Each tile of the board may be trapped according to a specific layout. In addition, the player have the choice of a dice to roll. Each specific dice may imply different consequences, triggering a trap that may takes you back to the starting tile, bringing you three steps back... Also, the game may be won according to Two rules that has to be predefined. Either you have to stop on the 15th tile (in the case of a circular board), if you overshoot, you are taken back to the start. Either you have to pass the 15th. The aim is to win the fastest possible.

1.2. Modelisation

We modeled this problem as a Markov decision process and we found the best strategies by implementing the value-iteration algorithm described in the course.

In this Markov decision process, we had a reward equal to 1 at each iteration if the initial state was not the final state. With this weighting strategy we could define the minimum expected value as the minimum number of turns before reaching the final state.

More formally, we define :

- $k \in S$ the states of the Markov decision processes, each state is represented as a tuple $k = (position, blocked)$ where $position \in \{1, \dots, 14\}$ and $blocked \in \{True, False\}$
- d the final state when we reach $position = 15$
- $U(k)$: the set of all possible actions available at state k . (In our case, the different dices we can use when $blocked = False$ and the action of passing the turn otherwise)
- $p(k' | k, a)$: the probability of reaching state k' from state k with action a .
- $c(a | k)$: the immediate reward (or cost) when performing an action. It is equal to 1 for any state.
- $V(k)$: the minimum expected total reward (or expected total cost), thus here the minimum number of turns before reaching the end state d from state k

$$\begin{aligned} - V(k) &= \min_{a \in U(k)} \{c(a | k) + \sum_{k'} p(k' | k, a) V(k')\}, k \neq d \\ - V(d) &= 0 \end{aligned}$$

To find the minimum total reward, we implemented the value iteration algorithm as such :

$$\begin{cases} \hat{V}(k) \leftarrow \min_{a \in U(k)} \left\{ c(a | k) + \sum_{k'} p(k' | k, a) \hat{V}(k') \right\}, k \neq d \\ \hat{V}(d) \leftarrow 0, \text{ where } d \text{ is the destination state} \end{cases} \quad (1)$$

After convergence, the best action (which dice to pick) is then provided by

$$\arg \min_{a \in U(k)} \left\{ c(a | k) + \sum_{k'} p(k' | k, a) \hat{V}(k') \right\}$$

1.3. Implementation

During our project, we implemented 5 different functions :

1.3.1. Markov decision

Function defined so that it satisfies the requirement given by the teacher. The results are given by the value iteration algorithm with parameters [PUT HERE THE FINAL PARAMETERS]

1.3.2. Value iteration

This function returns the best action and its expected value for each state based on the values given by the Expected Policy function.

1.3.3. Expected Policy

This is our main function, it is called by value-iteration to update the value of $\hat{V}(k)$.

1.3.4. Expected New Places

This function models each throw of a dice and returns a list with the new places and the probabilities of these new places after rolling the dice and encountering traps.

1.3.5. Transition

This function simulates a single turn in the game based on the current state, chosen action (dice type), board layout, and whether the game requires an exact roll to win. It returns the new position and whether the player should skip the next turn. This function differs from Expected policy such that Transition aims to simulate a single throw and expected policy wants to compute the expected value of a throw.

2. Results and Analysis

2.1. Optimal strategy depending on layout

Let's first observe some simple layouts.

In a game with no traps and no circle, the optimal strategy is to always pick dice 2. This is expected as there is no risk when there are no traps and no risk of going back to the start if we overshoot on the final roll. We also note that there is no strategy to try and take the shortcut.

With no traps and a circular layout, we observe that the optimal strategy is the same. However it takes precautions to not overshoot when getting close to the end.

With a layout full of restart, go back 3 steps and random traps. The optimal strategy is to pick dice 0 on every tile, expect on the first tile where dice 1 is used. Tile 1 because it's the start, if you restart it isn't a big problem. Otherwise it isn't worth it to risk a trap. We also note that no dice 2 is ever picked because of its 100% chance of triggering a trap, as we have a layout full of traps, it is a 100% chance of restart.

With only traps 3 (skip next turn). The optimal strategy consists in not missing the fast lane and then to only picking the second dice. Indeed, even if you skip always one turn, the expected value is still better then never taking risks.

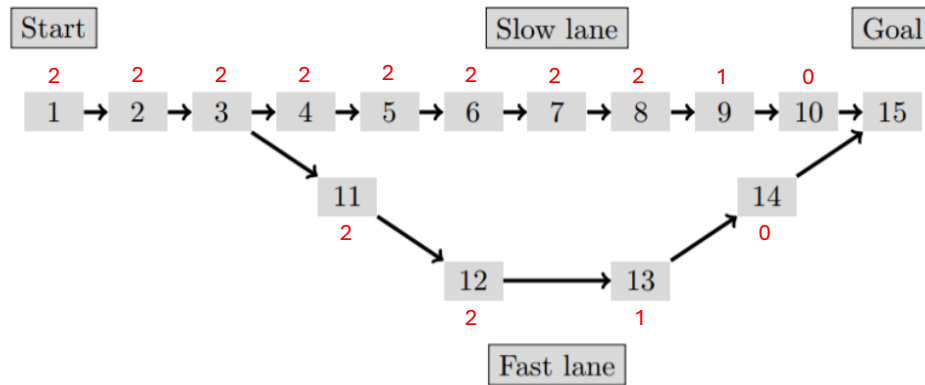


FIGURE 1 – Optimal strategy no traps + circular

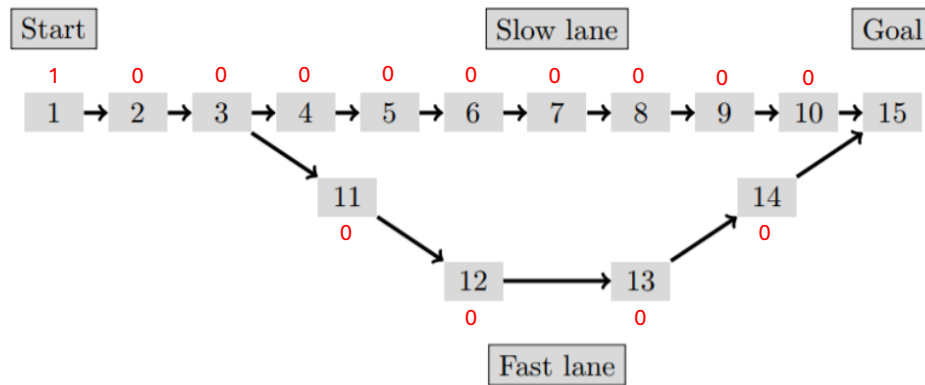


FIGURE 2 – Optimal strategy only traps 2 + no circle

With these models we discover that the overall best strategy is to not miss fast lane if possible, either by not missing it at first or either going back by falling into traps to try and take it. Furthermore, if the traps are too punishing, we simply play it safe and go slow.

2.2. Number of turns : expected vs experimental

Each simulation is 10 000 games following the optimal strategy, the experimental result is found by taking the mean of the number of turns, we also show the maximum and minimum number of turns taken

layout	circle	theoretical	exp average	exp max	exp min
no traps	false	6.6674	6.6478	19	3
no traps	true	7.2566	7.2808	24	3
only trap 1	false	17	17.0018	49	6
only trap 1	true	17	16.9213	47	6
only trap 2	false	17	16.9839	43	6
only trap 2	true	17	17.0013	47	6
only trap 3	false	11.5443	11.5355	34	4
only trap 3	true	11.8868	11.8912	35	4
only trap 4	false	16.50	16.5485	43	6
only trap 4	true	16.50	16.537	46	6

We observe that, with enough games played, the average number of turns is very close to what we predicted. Even with some lucky or unlucky runs.

2.3. Number of turns : other strategies

We define a less punishing layout by trapping 1/2 of all tiles with a trap of the fourth category. Otherwise, on trapped layouts, dice 2 will never be able to finish.

The layout is as following :(with the trapped cells in green and the optimal dice in red)

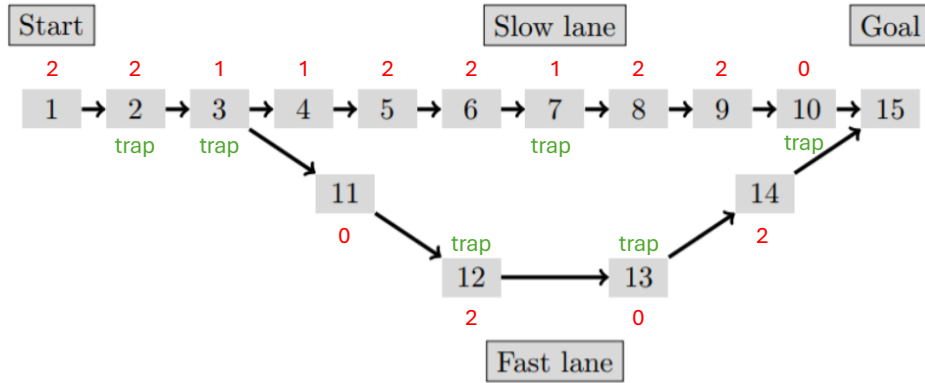


FIGURE 3 – Random layout + no circle used for simulations

We will compare the expected value against strategies of picking only dice 0, dice 1, dice 2 and picking a dice at random.

These simulations are over 10 000 games.

circle	theoretical	exp optimal	dice 0	dice 1	dice 2	random dice
false	13.3575	13.3716	17.0756	16.6936	15.4006	17.1171
true	13.4073	13.4302	17.0756	24.0166	25.481	25.5768

We observe that on a random layout, the strategy found by value iteration follows the expected theoretical value. We also notice that the "dice 0" strategy works the same way if the board is circular or not, which is expected because it doesn't trigger any traps nor overstep the finish tile.

On a non-circular board, The first and the second dice seem to work similarly. However, on a circular board we see a big jump in time caused by multiple oversteps. The random dice performs the worst among all three dices.

3. Conclusion

Finally, we can say that the aim of the optimal strategy is designed to play it safe if the traps are further along the board. Indeed, as we have no idea how the traps will behave, our player seems to be risk averse and decides only to play small moves that will make him move slowly but surely.

"Slow is smooth, and smooth is fast"

We have also proved that the optimal strategy is better than just choosing the same dice all the time or playing at random.

4. Part II

4.1. Introduction

In this part, we shall examine the game in the context of a competition.

Once two players are competing against each other to finish before the other, does the optimal strategy from a single-player game still hold? Considering both players aim to win, is the optimal strategy the one that minimizes the time to reach the final state, or is it one that ensures victory regardless of that? This strategy might be the most risk-averse or, conversely, the most risk-seeking.

In a real-life context, we could see this as two firms competing for the greatest market share. Will both firms adopt an aggressive strategy, investing heavily in advertising? Will the leading company, alternatively, choose to play it safe if the other is lagging in market share, or will it aggressively expand its lead?

This exploration provides insights into the dynamic interplay between competitive strategies, risk management, and decision-making in high-stakes environments.

4.2. Modelisation of the problem

In a two player setting, the snakes and ladder game will be modeled a little bit differently than in the one-player setting.

Because there are two players (player 1 and player 2), the states need thus to represent the position of both players in the snakes-and-ladders board. It also needs to take into account to whose player's turn it is. And finally whether one player has activated a skip your turn-trap or not.

Following these baseline, we are still able to model the problem as a Markov decision process.

We decided that the total expected reward would be the probability of player 1 to win the game (ie. reach the 15th tile before player 2). To achieve that, the immediate reward at one state will be 0, but the total expected reward for final states where player 1 wins will be equal to 1.

More formally, we have :

- S , the states of the Markov decision processes, each state is represented as a tuple, $k = (place_1, place_2, t)$, where $t \in \{1, 2\}$ denotes whose turn it is to play. $Place_i = (position_i, blocked_i)$, where $position_i \in \{1, \dots, 14\}$ and $blocked_i \in \{True, False\}$.
- d_1/d_2 the final states corresponding to when player 1/player 2 reaches the destination $position = 15$ before the other player.
- $U(k)$: the set of all possible actions available at state k . (In our case, the different dices the player whose turn it is, can be used when $blocked = False$, and the action of passing the turn when $blocked = True$)
- $p(k' | k, a)$: the probability of reaching state k' from state k with action a .
- $c(a | k)$: the immediate reward when performing an action, which is here equal to 0.
- $V(k)$: the expected total reward, thus here the expected probability of player 1 to win from state k .
 - $V(k) = \max_{a \in U(k)} \sum_{k'} p(k' | k, a) V(k') \quad \forall k \neq d_1, d_2 \mid t = 1$
 - $V(k) = \min_{a \in U(k)} \sum_{k'} p(k' | k, a) V(k') \quad \forall k \neq d_1, d_2 \mid t = 2$
 - $V(d_1) = 1$
 - $V(d_2) = 0$

To solve this, we implement a modified version of the value iteration algorithm seen in part I .

$$\begin{cases} V(k) \leftarrow \max_{a \in U(k)} \sum_{k'} p(k' | k, a) V(k') & \forall k \neq d_1, d_2 \mid t = 1 \\ V(k) \leftarrow \min_{a \in U(k)} \sum_{k'} p(k' | k, a) V(k') & \forall k \neq d_1, d_2 \mid t = 2 \\ V(d_1) \leftarrow 1 \\ V(d_2) \leftarrow 0 \end{cases} \quad (2)$$

After convergence, in case of convergence, and assuming that the game doesn't go on indefinitely, the probability that player 1 win is equal to the 1-probability that player 2 win.

Thus searching for a strategy that reduces the probability of winning of player 1, player two found the strategy that maximized his probability of winning.

4.3. Intuition on algorithm 2

If we initialize $\hat{v}(k) = 0 \quad \forall k$, we can interpret algorithm 2 in a special way.

At the first iteration, all value will be updated to 0 (because all V are 0), with the exception for the state d_1 . This can be interpreted as the probability of win of player 1 after 0 turns played.

At the second iteration, for all the states $\hat{v}(k) = 0$ but the states where $\exists a : p(d_1 | k, a) > 0$ and d_1 . Because for these states $t = 1$, the algorithm will try to maximise the probability of player 1 to win ... in 1 turn.

At the third iterations, for states where $t = 1$, the algorithm will still try to maximise the probability of player 1 to win in 1 turn because for all others states. $k' : p(k' | k, a) > 0, t = 2$ and those are still equal to zero.

For all states where $t = 2$, the algorithm will try to minimize the probability to go in states where $V(k') > 0$. In some states, where the position of player 1 is close to 15 (and he skip not next turn), this means trying to maximize the probability to go to d_2 because it is the only state with $V(k') = 0$.

At the fourth iteration for states where $t = 1$, the algorithm will try to maximise the probability of player 1 to win in 2 turn and so on and so on.

We see that every 2 iteration, we simulate 1 turn of play and update the probability of winning of player 1 in a game of n turns for each state knowing the future probabilities of win of player 1 in a game of $n-1$ turns.

4.4. Min-Max and Max-Min

We have seen that our algorithm has player 1 trying to maximise his probability of winning while player 2 tries to minimise player's 1 probability V_1 , but what if we tried to make the inverse? Using player's 2 probability to win V_2 ? Will that change the result?

If we assume the game always ends in a final state, d_1 or d_2 , then the probability of player 2 to win with our setting becomes 1 - probability of player 2 to win.

We can then define two utility values, $u_1(k) = V_1(k) - 0.5 = 0.5 - V_2(k) - P(k)$ and $u_2(k) = 0.5 - V_1(k) - P(k) = V_2(k) - 0.5$ where $P(s)$ is the probability the game does not ends from state k . If both players try to maximise there utility, and $P(k) = 0$, then we have a 0-sum game.

But in a 0-sum game, $\min_2 \max_1 u_1(k) = \max_2 \min_1 u_2(k) \implies V_1 = V_2$. (NB : in our case when one players plays, the other does not have any choice of moves).

We can calculate $P(k)$ using what we will call "twin states" (k_1 and k_2), which are states where both player are in the same place but in one state it is the turn of player 1 to play and on the other the turn of player 2.

We can then estimate $\hat{P}(k_1) = \hat{P}(k_2) = \hat{V}_1(k_1) - \hat{V}_1(k_2)$

In all runs, we observed $\hat{P}(k) \rightarrow 0$, But there are scenarios where the risky strategy leads to never go to the end from some positions (when all the position have the trap 1 or 2). But is is trivial to see that this strategy will never be chosen because never optimal (NB : in a game where player 1 needs to minimize V_1 and player 2 to maximize it, then we can not prove $V_1 = V_2$ anymore because of this configuration).

If not for these strategy, all combinations of strategy will have a non-0 probability to reach one of the end states at some points, and that will lead to a markov chain with only those two end states as terminal states and thus $P(k) \rightarrow 0$ where we run algorithm 2 infinitely many times.

4.5. Results and Analysis

We will test the optimal strategy on non-circular layouts with different traps. For these following results, we will be in the perspective of the player that starts the game.

For a layout without any traps, we observe that the strategy differs at the start. If the other player is further in the board, we will try to take the shortcut at tile 3. We also observe a divergence on tile 8, where player prefers dice 1 over dice 2.

In the following table, we show where our player is, where the other player is, the best move provided by min-max algorithm we implemented and the best move provided by value-iteration. We show only the states where min-max makes a different decision to the value-iteration algorithm.

our tile	other tile	min-max dice	solo dice
1	10	1	2
1	14	1	2

For the random layout described in figure 3, we observe that in general, when our player is lagging behind, it will take more risks to try and make up for his disadvantage. Similarly, when our player is in front, it will take the less risky dice to make sure it doesn't fall into a trap.

our tile	other tile	min-max dice	solo dice
1	10	1	2
1	14	1	2
2	10	0	2
2	13	0	2
2	14	0	2
3	1	0	1
3	2	0	1
8	1	0	2
8	2	0	2
8	3	0	2
8	4	0	2
9	1	0	2
9	2	0	2
9	3	0	2
9	4	0	2
9	5	0	2
9	6	0	2
9	7	0	2
9	11	0	2
10	8	2	0
10	9	2	0
10	10	2	0
10	13	2	0
10	14	2	0
11	7	2	0
11	13	2	0
11	14	2	0
12	1	0	2
12	2	0	2
12	3	0	2
12	4	0	2
12	5	0	2
13	8	2	0
13	9	2	0
13	10	2	0
13	12	2	0
13	13	2	0
13	14	2	0

We observe that generally, the player takes less risk when the other one is behind and more risks when the other is closer to the finish tile. This results in a longer game than optimal but here the goal is to arrive before the other, not necessarily with the fastest time.

The expected number of turns to reach the end on this layout with value-iteration is 13.357 turns, whereas the min-max expects 13.59 turns. We compute this expected number of turns with the action chosen by min-max in the same way as in value-iteration. When we are in a state where the other player won, the expected number of turns is directly given by the number of turns of value-iteration.

With the min-max algorithm, we determined that there is a slight bias towards the player who starts, as he has a 52% chance of winning the game, whereas player 2 has a 48% chance of winning.

Indeed, as he is the first player to throw the dice, he can make his opponent play in a riskier way which may cause him to fall into traps more often.

4.6. Conclusion

These different strategies are interesting in a business world. We can draw an easy link between this, "I have to go faster because people are ahead of me" or "I can slow down, because people are behind me" and the "Barriers to entry" concept. Indeed, one can define barriers to entry as a high startup costs or other obstacles that prevent new competitors from easily entering an industry or area of business. It can be high costs, legal barriers, economy of scales or mastering technologies. Those barriers allow installed firms to develop new products instead of focusing on the competition. In our previous context, one can define those barriers to entry as the advances that one player has over the other one. However, those barriers to entry can be broken, in a business context, it's often recommended to rethink the market you are wanting to enter. Here, the breaking occurs through the risk. Indeed, since the only way of winning the game, is to finish ahead of your opponent, the only available option you have is to play it risky. We could eventually, remodel the game so that new ways of breaking those barriers could be implemented.

Further consideration

In order to go further, we could potentially try to revise this problem into a multiple player game. The objective, while still winning, would probably differ as, only one player will be the grand winner. However, for the rest of the players, the competition will continue. As we've seen in the two player context, the second player, if he is behind the first, will play in a risky way knowing that he might be falling into a trap. If we consider a third player that is behind the second, will the second play risky in order to catch the first, or will he play safe in order to lower his risk of falling into a trap and being caught by the third.